

Need ideas for the following conference:

<https://vlsid.org/call-for-udt/>

Need something groundbreaking and disruptive that matches the requirements. And gives us a more than clear shot to get selected.

don't worry about submission date. Need a new developed idea.



I've checked the actual UDT criteria. The key is not merely "a cool chip idea" — it needs a **buildable design methodology + hard numbers + a disruptive story** that fits silicon-to-smart-systems.

I'm going to bias toward ideas that you can actually prototype from RTL/FPGA/open-source EDA, rather than proposing a billion-dollar fabrication project that looks impressive only on paper.

Yes. I checked the **actual VLSID 2027 UDT call**, and I think we should approach this very differently from a normal student conference project.

The UDT explicitly rewards **originality, disruptive innovation, practical applicability, scalability, and measurable results**. It also explicitly accepts **work-in-progress**, prototypes, AI-assisted design methodologies, EDA automation, hardware security, intelligent systems, genomics-on-chip, agentic AI, etc. ([VLSID 2027](#))

One important warning: **there is no idea that gives a guaranteed selection**. But we can absolutely engineer a project that is unusually well aligned with what the reviewers are looking for.

And after looking at what has already appeared around VLSID and the 2025–26 literature, I would **not** do another generic:

- RISC-V processor
- CNN accelerator
- AI image classifier on FPGA
- IoT system
- "LLM generates Verilog"
- generic neuromorphic accelerator

Those are too easy to look like incremental student projects.

The direction I would take

I see **three genuinely strong candidates** for you.

Ran k	Idea	Novelt y	Buildabilit y	UDT fi t	My confidenc e
	Autonomous Self-Healing SoC	9/10	8/10	10/10	Very high
	Proof-Carrying AI Hardware Compiler	9/10	9/10	10/10	Very high
	Adaptive Bio-Diagnostic SoC	10/10	6/10	9/10	High
4	Cognitive / self-optimizing RISC-V	8/10	8/10	9/10	High
5	Runtime-morphing AI accelerator	8/10	6/10	9/10	Medium-high

But **#1 and #2** are where I'd put serious effort.

1. "The Silicon Immune System"

Autonomous Self-Healing RISC-V SoC

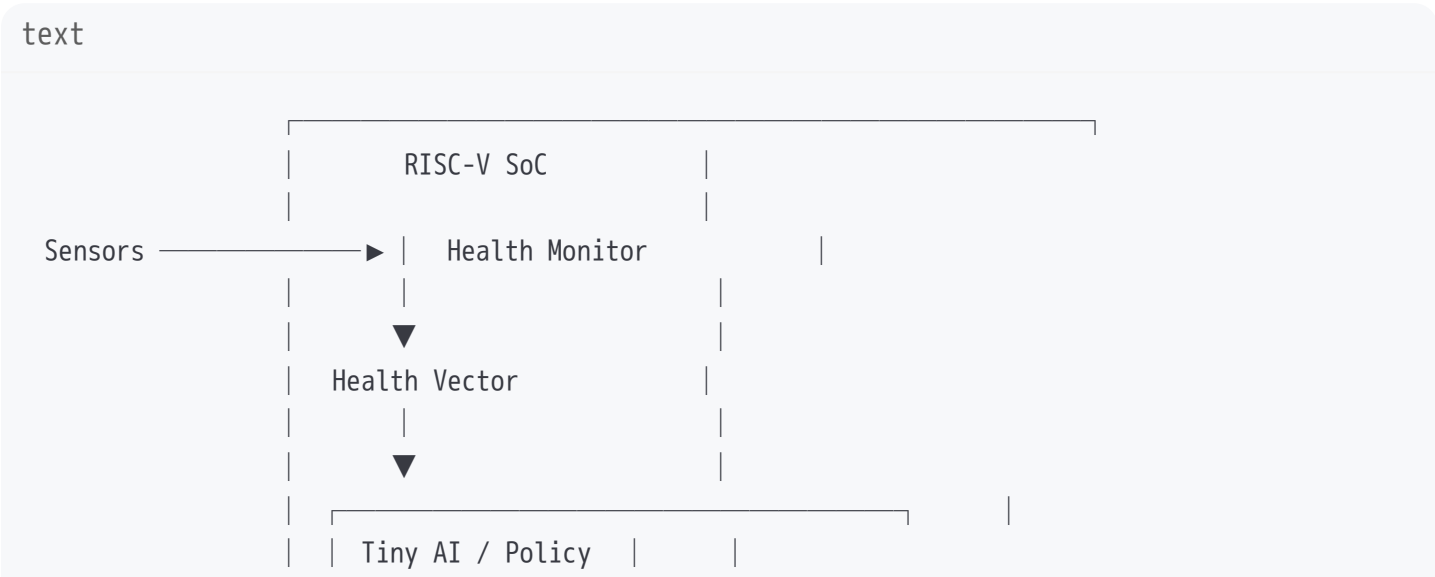
This is the one I'd seriously consider building.

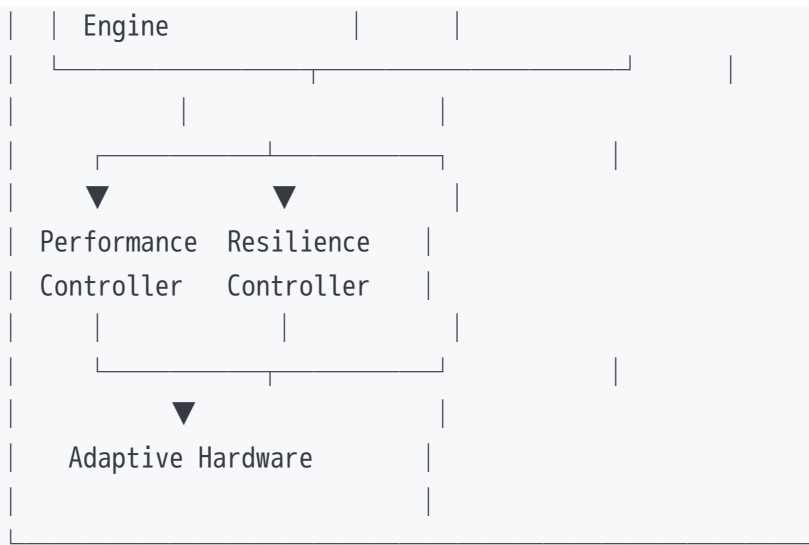
The basic philosophical shift is:

Instead of designing hardware that assumes everything will remain healthy, design hardware that continuously monitors itself, diagnoses its own degradation, and changes its behaviour to remain operational.

Think of it as an **immune system for silicon**.

Proposed architecture





The SoC continuously observes:

- temperature
- timing margin
- error rate
- memory errors
- workload intensity
- voltage/frequency state
- execution anomalies
- injected faults

Then it calculates a **silicon health vector**.

For example:

text

```
Health =  
[thermal=82,  
 timing=71,  
 memory=94,  
 security=100,  
 workload=37]
```

The hardware then decides:

"I'm getting thermally stressed. Reduce frequency."

Or:

"The workload changed. Switch to low-power configuration."

Or:

"Memory errors are increasing. Enable stronger protection."

Or:

"This execution unit is behaving abnormally. Route around it."

But here's the part that makes it interesting.

Don't make it merely:

sensor → threshold → action

That's ordinary adaptive hardware.

Instead:

Give the SoC multiple "survival policies."

For example:

Mode 1 — Performance

text

high frequency
low redundancy
large cache
aggressive execution

Mode 2 — Energy

text

frequency ↓
voltage ↓
cache ↓
aggressive clock gating

Mode 3 — Reliability

text

redundancy ↑
ECC ↑
frequency ↓
fault isolation ↑

Mode 4 – Survival

text

minimum functionality
faulty block bypass
reduced performance
maximum reliability

The **on-chip policy engine chooses the appropriate configuration automatically.**

That gives you a much better paper:

"A Self-Aware RISC-V SoC That Dynamically Trades Performance, Energy and Reliability to Preserve System Operation Under Degradation."

Why I really like this for VLSID 2027

Look at their theme:

Artificial Intelligence → Physical Intelligence

and:

Compute → Cognition → Silicon → Smart Systems

Your chip literally **senses → reasons → acts on its physical operating condition.**

That's almost tailor-made for the conference. (VLSID 2027)

It also hits multiple UDT categories simultaneously:

- RISC-V
- low-power design
- reliability
- hardware security
- intelligent systems
- AI at the edge
- adaptive computing
- cyber-physical systems
- intelligent semiconductor design

And we can actually build this.

You don't need silicon fabrication.

Build a complete RTL prototype.

Hardware

- RV32I/RV32IM core
- configurable cache
- ECC-protected memory
- watchdog
- performance counters
- thermal proxy
- timing-stress monitor
- fault injector
- health monitor
- adaptive controller

AI

Don't put a giant neural network inside.

Use something tiny:

- decision tree
- contextual bandit
- tiny MLP
- Q-learning
- lightweight anomaly detector

The intelligence itself becomes a VLSI contribution.

The killer experiment

This is where the project becomes UDT-worthy.

Run identical workloads under controlled degradation.

Baseline

Normal RISC-V:

text

Performance	100%
Power	100%
Fault tolerance	X
Availability	X

Then inject:

text

temperature stress
timing violations
SEUs
memory corruption
workload changes

Compare:

Static architecture

versus

Your self-healing architecture

Measure:

- performance
- energy
- recovery latency
- number of detected faults
- number of recovered faults
- availability
- area overhead
- frequency
- power
- fault coverage

Your slide 5 can then contain something like:

94.7% of injected faults recovered autonomously with only 6.8% area overhead and 4.2% average performance degradation.

Obviously, **those numbers are examples**, not claims. You would generate the actual numbers experimentally.

That is exactly the kind of quantitative evidence UDT asks for. ([VLSID 2027](#))

One problem

Someone has already published a 2026 IEEE TNS paper on a **self-healing RISC-V soft processor using dynamic partial reconfiguration**, including fault injection and proton irradiation. ([ResearchGate](#))

So we **must not simply reproduce that**.

Our novelty would instead be:

Existing

```
text

fault
  ↓
detect
  ↓
recover
```

Our system

```
text

physical telemetry
  ↓
health state estimation
  ↓
AI/policy reasoning
  ↓
performance / power / reliability tradeoff
  ↓
microarchitectural adaptation
  ↓
continuous learning
```

That's a much broader concept.

2. Proof-Carrying AI Hardware

This one might actually be **even more strategically clever**.

The problem:

AI can generate RTL.

But:

Who trusts AI-generated hardware?

LLMs can generate syntactically valid Verilog that is functionally wrong, inefficient, insecure, or physically terrible.

Recent work is already moving toward closed-loop agentic EDA, PPA-aware RTL generation, and tool-integrated multi-agent hardware design. ([DOI](#))

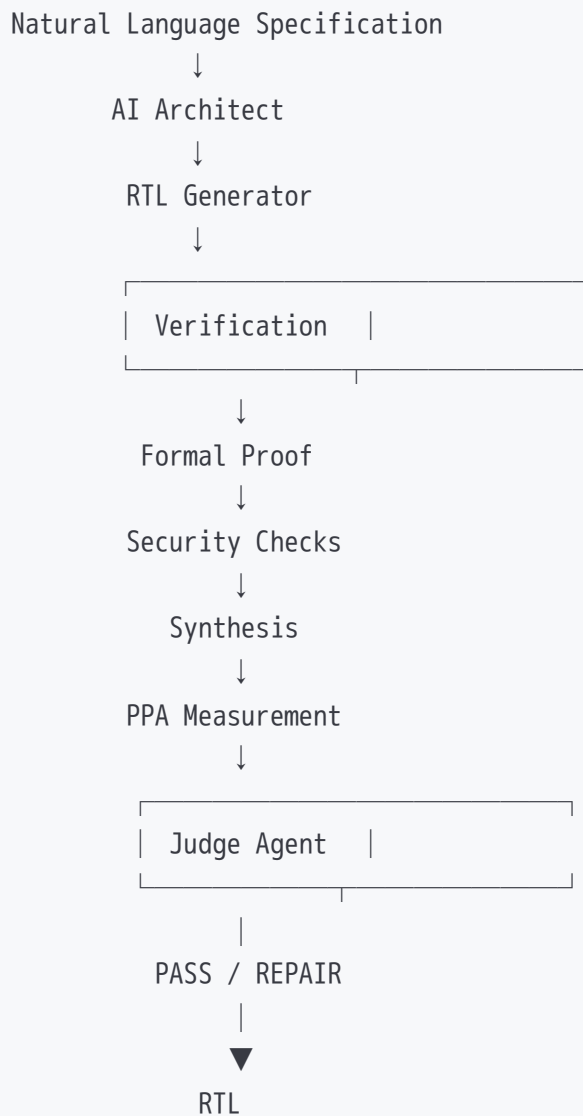
So don't build another "AI writes Verilog" project.

Build:

An AI Hardware Compiler That Cannot Submit Unverified RTL

Concept:

text



But here's your killer idea:

Every generated hardware block gets a "Design Passport."

Something like:

text

HARDWARE PASSPORT	
Functional correctness:	PASS
Formal properties:	47
Coverage:	98.3%
Security checks:	PASS
Timing:	PASS
Area:	12.4k
Power:	8.7mW
Critical path:	3.2ns
Equivalence:	PASS

The AI isn't allowed to say:

"I think this RTL is correct."

The **EDA toolchain must prove it.**

That is a much more mature proposition.

And we can make it even more interesting.

Introduce a concept called:

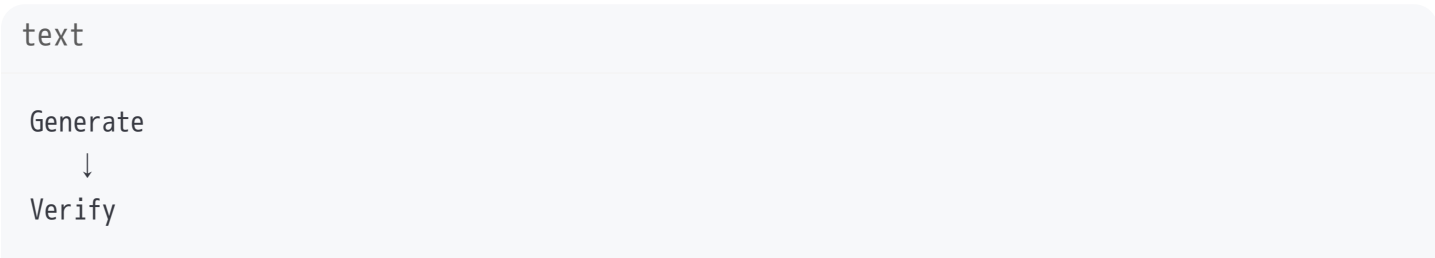
"Proof-before-optimization"

The AI cannot optimize RTL unless:

- 1. functional correctness is established
- 2. assertions pass
- 3. formal equivalence passes
- 4. security checks pass

Then it can optimize PPA.

So:



↓
Prove
↓
Optimize
↓
Re-prove
↓
Accept

This directly addresses the "tool-feedback gap" that current agentic hardware research identifies. ([DOI](#))

And you can build this almost entirely with open-source tools.

Something like:

text

```
Python
|
|—— LLM
|
|—— Verilog generator
|
|—— Verilator
|
|—— Yosys
|
|—— OpenROAD
|
|—— SymbiYosys
|
|—— fault/security analyzer
```

You could test:

- ALUs
- FIFOs
- arbiters
- UART
- AXI/APB blocks
- RISC-V modules
- cache controllers
- cryptographic modules

And benchmark:

Human RTL

vs

Standard LLM

vs

Your system

Metrics:

text

functional correctness
formal proof rate
PPA
number of iterations
time to convergence
bugs detected
security violations

That is **extremely conference-friendly**.

3. 🏆 The wild card: Bio-Diagnostic SoC

And this one is interesting because it connects directly to the kind of work you've already been exploring.

But I'd completely rethink the project.

Don't make:

"Lab-on-chip using Arduino."

That's nowhere near VLSID UDT level.

Instead:

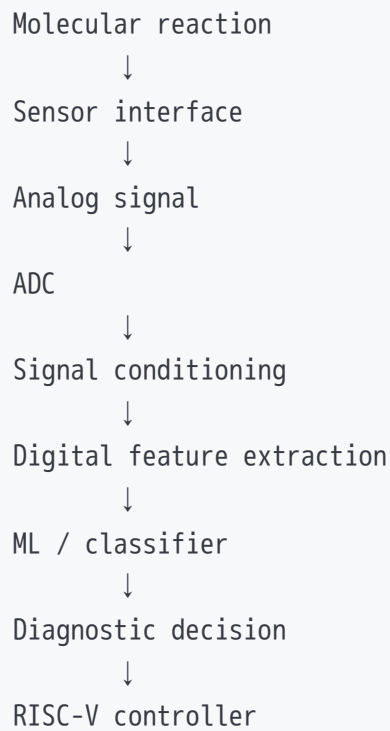
A Reconfigurable Molecular Diagnostic SoC

The idea:

A hardware architecture that converts molecular-test signals into real-time diagnostic decisions at the edge, without requiring cloud computation.

Think:

text



The interesting part is making the architecture **reconfigurable for different assays**.

For example:

text

LAMP

|
|—— temperature profile
|—— fluorescence characteristics
|—— amplification curve

CRISPR

|
|—— fluorescence
|—— threshold
|—— reaction kinetics

One SoC.

Different biological tests.

But don't stop there.

Create a:

"Molecular Signal Instruction Set"

Instead of processing everything in software, introduce specialized instructions such as:

```
text
AMP_DETECT
CURVE_FIT
NOISE_REJECT
THRESHOLD_EST
KINETIC_CLASSIFY
RESULT_CONF
```

Then the RISC-V processor gets a tiny molecular-signal accelerator.

Now you're not doing a biology project.

You're doing:

domain-specific computer architecture for molecular diagnostics.

And VLSID explicitly lists **bio-inspired and genomics-on-chip technologies** among its emerging technology topics. ([VLSID 2027](#))

This could be genuinely unusual.

But this one has a major weakness

You need convincing biological data.

You could use:

- public datasets
- experimentally generated synthetic traces
- literature-derived curves
- a professor's existing lab data

But the validation story is harder than the first two.

So I'd put it behind the first two.

4. A fourth idea I really like

"MorphCore"

A RISC-V processor that changes its own microarchitecture according to workload.

Instead of:

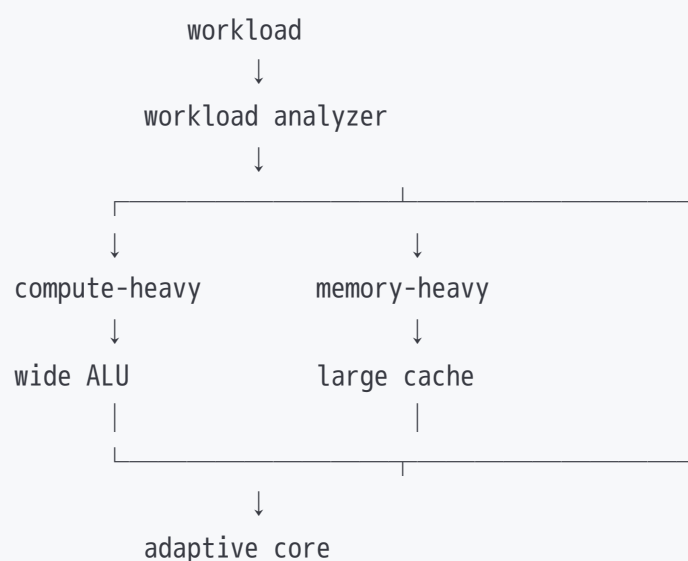
```
text

one processor
↓
```

all workloads

you create:

text



The processor has configurable:

- cache size
- pipeline depth
- issue width
- execution units
- branch predictor
- precision
- accelerator activation

And a tiny ML controller decides the configuration.

The research question becomes:

Can hardware dynamically trade architectural complexity for workload-specific efficiency?

That is stronger than another RISC-V CPU.

5. The absolutely crazy version

If you want something that could potentially become your **flagship project**, combine #1 + #4.

EvoSoC

A self-evolving RISC-V SoC

The processor has:

perception

text

workload
temperature
errors
power
timing

cognition

text

tiny ML policy engine

action

text

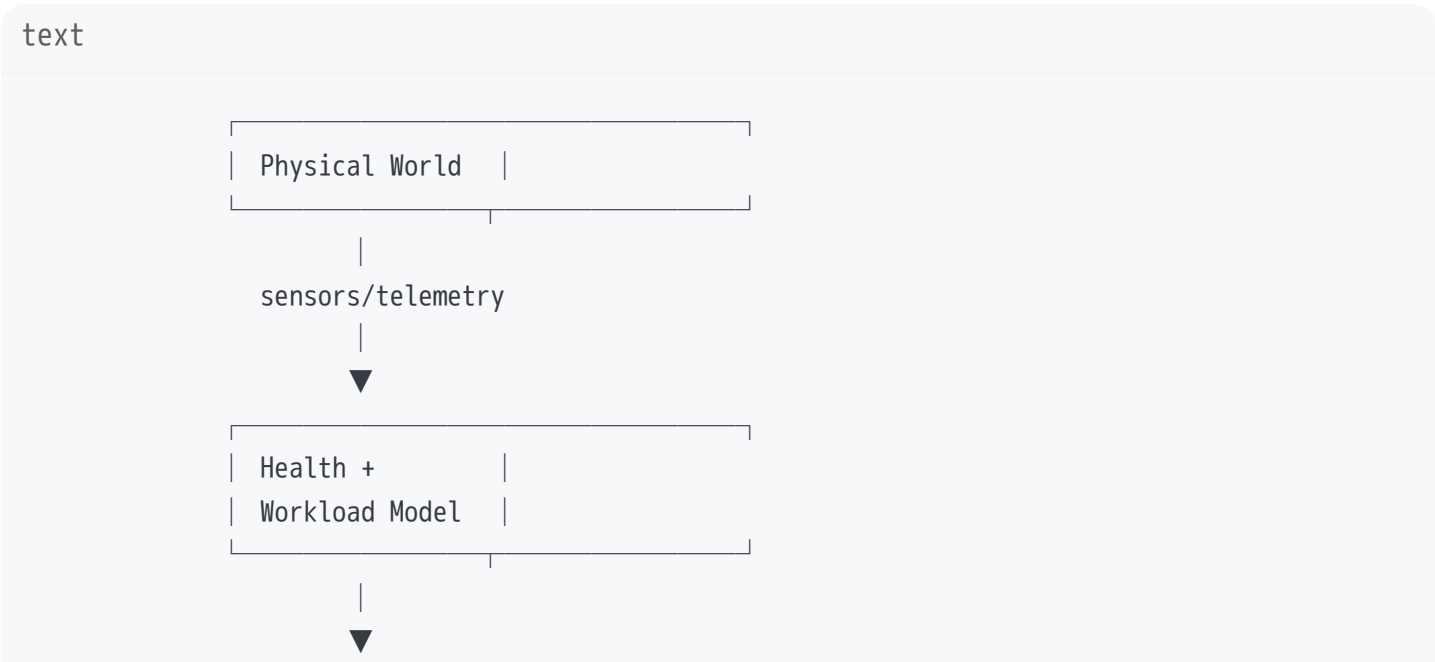
change architecture

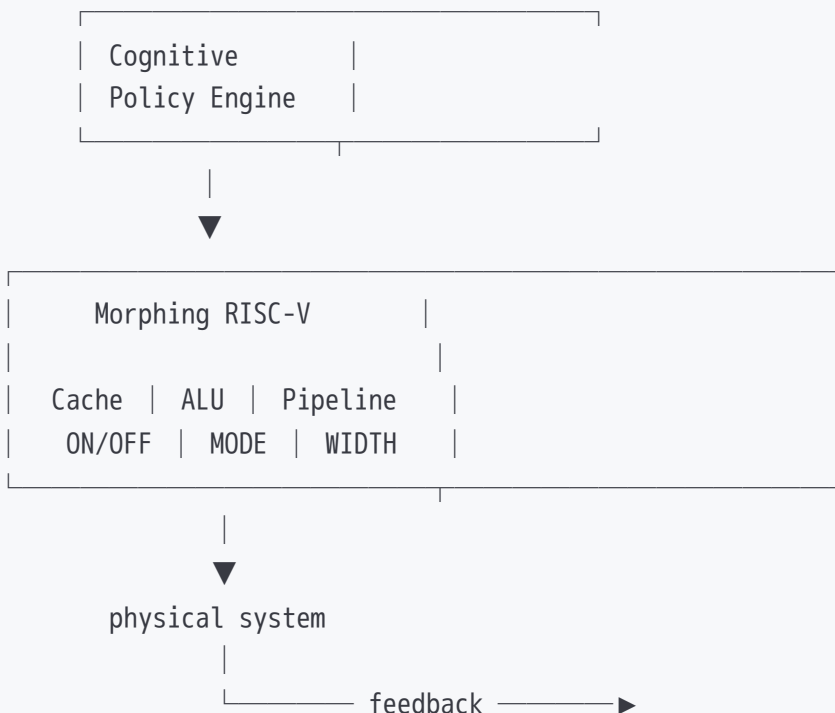
memory

text

learn from previous workloads

So:





The thesis:

Today's processors are designed once and operated many times. EvoSoC proposes processors that continuously adapt their architecture to the conditions in which they operate.

That is a **much more powerful story** than "we designed a RISC-V processor."

What I would NOT do

I'd kill these ideas immediately:

❌ Generic AI accelerator

There are thousands.

❌ RISC-V + CNN

Too ordinary.

❌ FPGA object detection

The 2026 UDT already had an FPGA-accelerated AI assistive device selected, so you'd be entering an increasingly populated category. ([LinkedIn](#))

❌ "AI-powered EDA"

Too vague.

❌ LLM generates Verilog

Already heavily researched.

Recent work includes PPA-aware LLM/RL hardware generation, test-time RTL optimization, multi-agent RTL systems and EDA-in-the-loop generation. ([IEEE Xplore](#))

✗ Another neuromorphic accelerator

Interesting, but the field is crowded and recent RISC-V/neuromorphic systems already demonstrate substantial FPGA implementations. ([ScienceDirect](#))

My actual recommendation

If I were on your team, I'd choose:

🏆 EvoSoC / Silicon Immune System

but I'd structure it as a **design methodology**, not merely a processor.

The title could eventually become:

"EvoSoC: A Self-Aware and Self-Adaptive RISC-V Architecture for Autonomous Performance, Energy and Reliability Management"

And the central contribution would be:

1. Silicon health model

A unified representation of:

text

thermal
timing
power
fault
memory
workload

2. Cognitive policy engine

Tiny ML model deciding what to do.

3. Morphable hardware

Hardware that can actually change operating configuration.

4. Fault-injection framework

Systematically inject:

text

SEU
memory corruption
timing stress
thermal stress
workload spikes

5. Closed-loop evaluation

Compare:

text

STATIC RISC-V
vs
ADAPTIVE RISC-V
vs
EvoSoC

6. Hard numbers

And this is **non-negotiable**.

We want to walk into the UDT review with a table like:

Metric	Static	Adaptive	EvoSoC
Energy	100%	82%	68%
Performance	100%	96%	98%
Fault recovery	0%	74%	94%
Recovery latency	—	14 μs	4 μs
Area overhead	—	9%	13%
Availability	91%	97%	99%

Again, those values are **targets/example formatting**, not fabricated results.

Why this could be a killer UDT submission

The UDT explicitly asks for:

problem → innovation → architecture/methodology → measurable impact → lessons/future work.
(VLSID 2027)

EvoSoC maps almost perfectly:

Slide 1

EvoSoC – Silicon That Adapts

Slide 2

Today's SoCs are statically optimized despite dynamically changing workloads and physical conditions.

Slide 3

Self-aware architecture.

Slide 4

Health vector + cognitive policy + morphable hardware.

Slide 5

Actual FPGA/RTL results.

Slide 6

"From programmable hardware to adaptive hardware."

That last sentence is the kind of framing I'd want.

One more strategic observation

VLSID 2027 is explicitly trying to move the conversation from **AI as computation** toward **AI interacting with the physical world**. ([VLSID 2027](#))

So don't ask:

"How can we put AI into a chip?"

Ask:

"What if the chip itself could perceive its physical state, reason about it, and change itself?"

That's a much more interesting research question.

And importantly, it gives you a project you can actually **build from RTL → FPGA → fault injection → quantitative evaluation**, rather than needing a 28-nm tapeout.

If we choose this direction, I would **not start making the PPT yet**. The next step should be to do a proper novelty sweep around **self-aware/self-adaptive SoCs, runtime architecture morphing, silicon health monitoring, fault recovery, tiny ML controllers, and RISC-V**, then design a specific architecture whose novelty we can defend against existing papers. That's where we can turn this from a cool idea into something genuinely difficult for a reviewer to dismiss.

I love the flagship idea of combining 1 and 4. Let's develop this further. We want something more groundbreaking. Something that completely changes how applicable industries work and function

 ChatGPT gpt-5-6

Absolutely. If we're going to pursue **EvoSoC**, I don't think we should frame it as "a smarter RISC-V processor." That's still too small.

The much bigger idea is:

What if computing infrastructure stopped being manufactured for a specific workload and instead became capable of continuously reorganizing itself around whatever the physical world demands?

That changes the target from **adaptive processor** → **adaptive computing infrastructure**.

I would push EvoSoC toward that.

Okay continue ideating, see if we can touch more features or ideas that support and enlarge the impact

 ChatGPT gpt-5-6

Yes. I think we should **expand EvoSoC from a processor into a new computing paradigm**.

The VLSID brief is unusually favorable to this: it explicitly calls for autonomous/adaptable/reconfigurable architectures, RISC-V, reliability, low-power systems, AI hardware, fault recovery, cognitive/adaptive/self-learning systems, smart infrastructure, industrial automation, and physical intelligence. ([VLSID 2027](#))

The trick is to make all of those **one coherent invention**, rather than bolting features onto a CPU.

The bigger idea: EvoSoC → "Living Computing"

The fundamental assumption of today's computers is:

Hardware is fixed. Software adapts.

We could challenge that.

Our proposition:

Hardware should adapt alongside software.

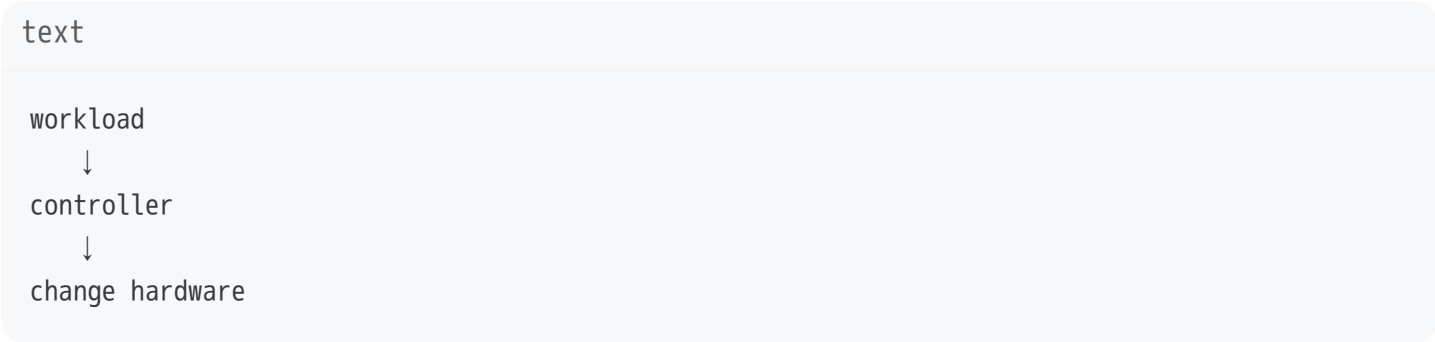
And eventually:

A computing system should continuously sense its environment, understand its own condition and workload, predict what it will need next, reorganize its resources, and learn from the outcome.

That gives us something much bigger than a self-optimizing processor.

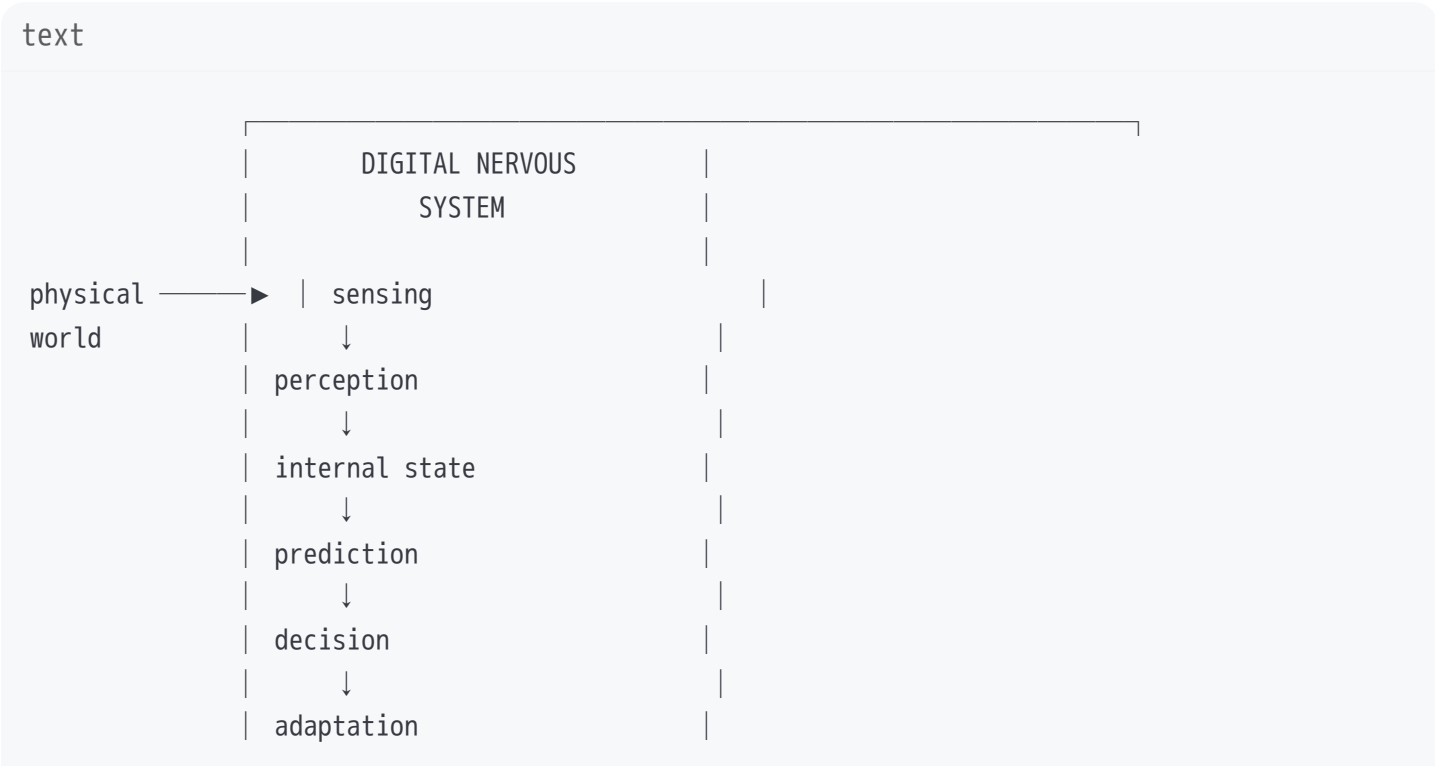
1. Give EvoSoC a "digital nervous system"

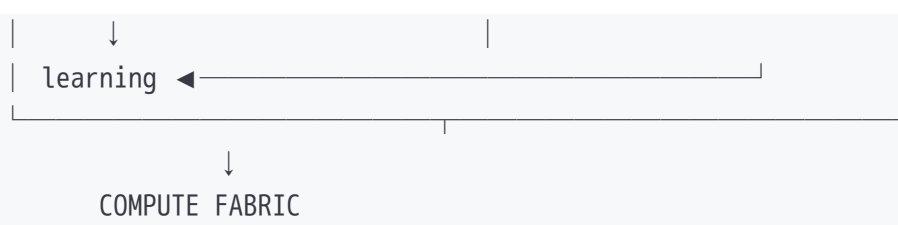
Right now we have:



Too simplistic.

Instead:





This gives us six layers:

① Sense

What is happening?

- temperature
- voltage
- current
- timing margin
- memory errors
- workload
- accelerator utilization
- network traffic
- sensor inputs

② Perceive

What does that mean?

text

```
"thermal stress increasing"  
"memory becoming unreliable"  
"AI workload incoming"  
"CPU underutilized"  
"latency becoming critical"
```

③ Predict

What happens next?

text

```
temperature likely to exceed limit  
workload likely to spike  
memory error probability increasing  
battery likely to enter low-power state
```

④ Decide

What should the chip do?

⑤ Adapt

Actually modify:

- frequency
- voltage
- cache
- pipeline
- accelerator allocation
- precision
- redundancy
- memory policy
- compute topology

⑥ Learn

Did the decision work?

If not:

modify the policy.

Now we have a **closed-loop physical intelligence system**.

And that phrase is almost tailor-made for VLSID 2027's "AI → Physical Intelligence" theme. ([VLSID 2027](#))

2. The biggest expansion: don't adapt only to the chip

This is where I think the idea becomes genuinely disruptive.

We currently have:

Self-aware chip

Let's make it:

Self-aware system.

Imagine EvoSoC controlling an industrial robot.

It doesn't just know:

"My CPU is hot."

It knows:

"The motor is under unusually high load, my compute demand is rising, vibration has increased, and the current thermal trajectory indicates a failure risk."

Therefore:

text

```
robot
  ↓
sensors
  ↓
EvoSoC
  ↓
predictive model
  ↓
change computation
  ↓
change robot behaviour
```

The hardware becomes part of the **control loop of the physical machine**.

That opens:

Automotive

The chip could dynamically trade:

performance ↔ safety ↔ energy ↔ thermal constraints

Robotics

Compute resources follow the robot's current task.

Drones

The architecture responds to:

battery + temperature + flight dynamics + mission priority.

Satellites

The architecture adapts to:

radiation + workload + thermal state + available power.

Industrial equipment

Compute allocation follows:

machine state + predictive maintenance + production requirements.

Medical devices

The system can prioritize:

safety and reliability over performance.

Now we're talking about a **platform technology**, not a CPU.

3. Introduce "Mission Mode"

This could be one of the coolest features.

Instead of applications merely running on hardware, the system receives a **mission objective**.

For example:

text

```
MISSION:
Maximize battery life
while maintaining
95% perception accuracy
and <10 ms latency.
```

Or:

text

```
MISSION:
Maintain robot safety.
Performance is secondary.
```

Or:

text

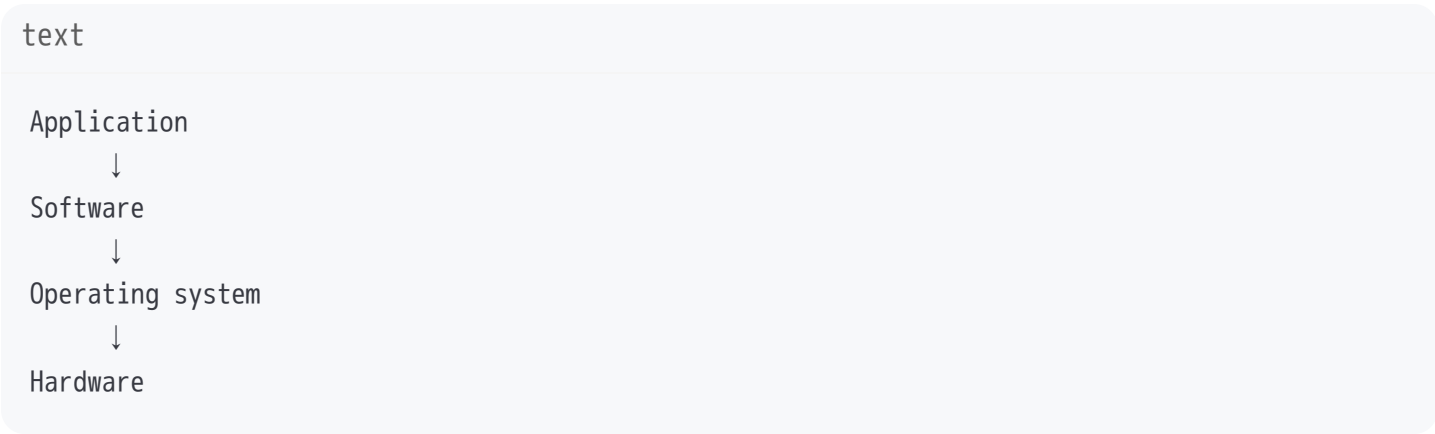
```
MISSION:
Maximum throughput.
Energy is secondary.
```

The EvoSoC controller then continuously searches for the hardware configuration that satisfies the mission.

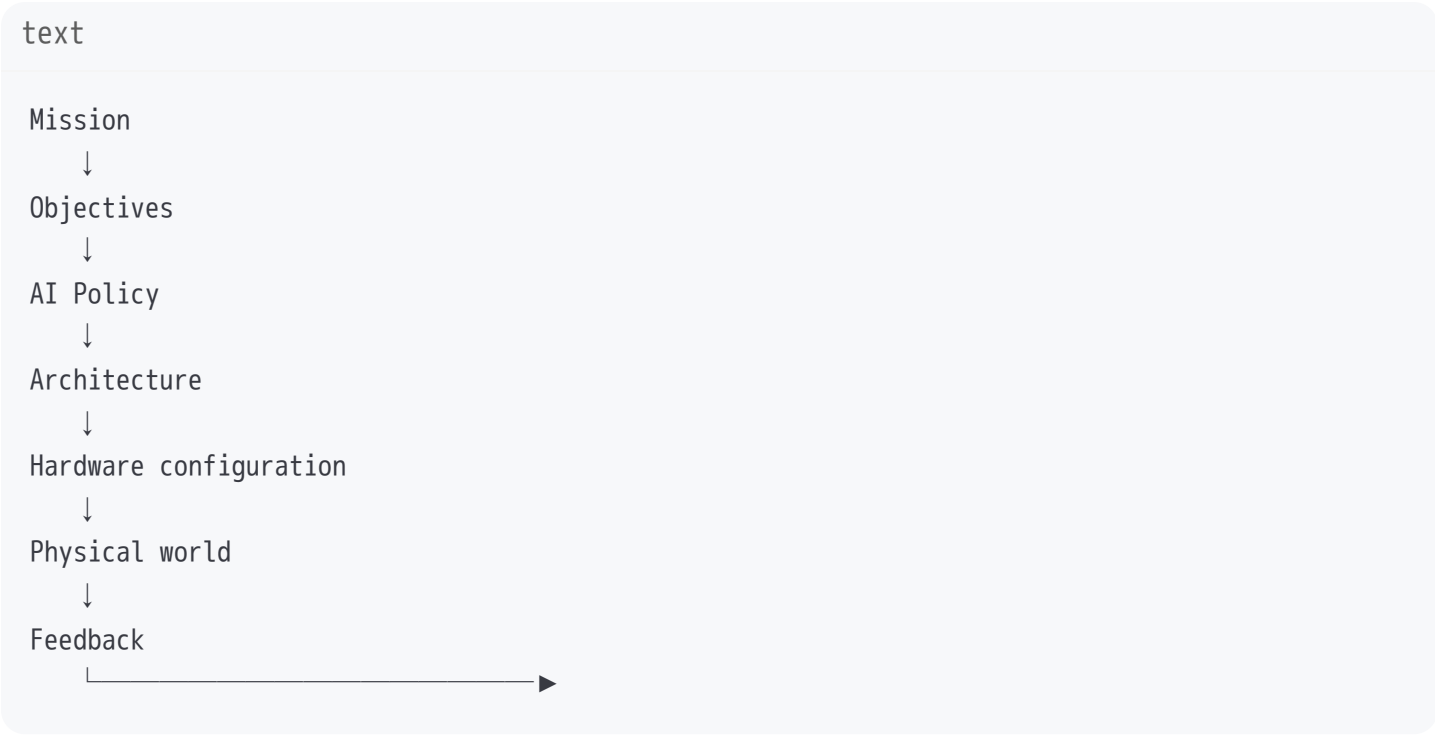
This creates a new abstraction:

Mission → Architecture

Today's stack:



EvoSoC:



That is much more radical.

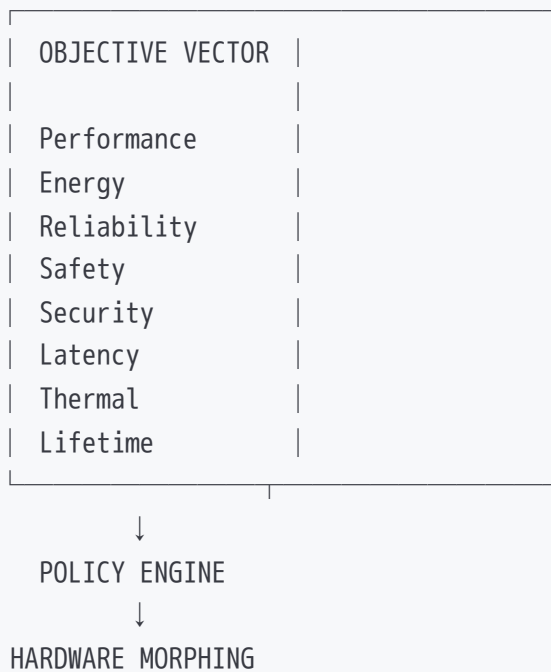
4. Give it multiple optimization dimensions

Don't make the AI optimize only PPA.

Give it a **multi-objective optimization engine**.

Something like:





So instead of:

"make it faster"

we can say:

"Find the safest configuration that meets 10 ms latency while minimizing energy and maintaining 99.99% availability."

That's a very different computer architecture philosophy.

5. Now add "resource trading"

This is where it gets really interesting.

Imagine the chip has:

text

4 CPU cores
2 AI accelerators
DSP
crypto engine
large cache
SRAM
reconfigurable fabric

Today's architecture:

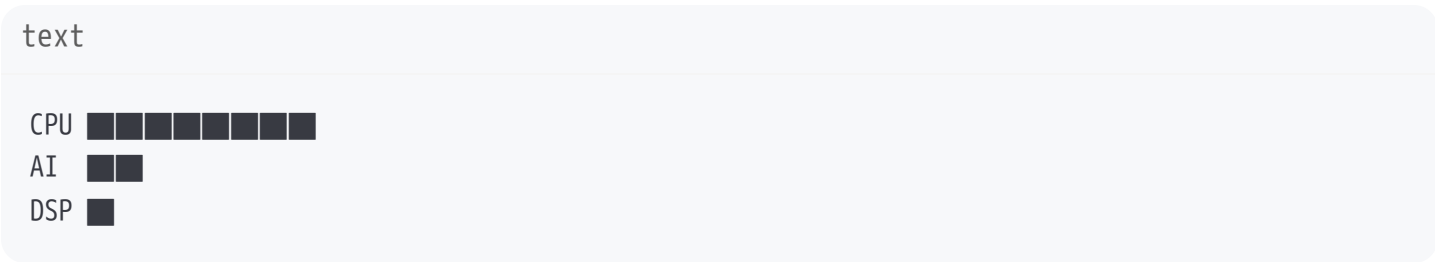
Everything exists whether you need it or not.

EvoSoC:

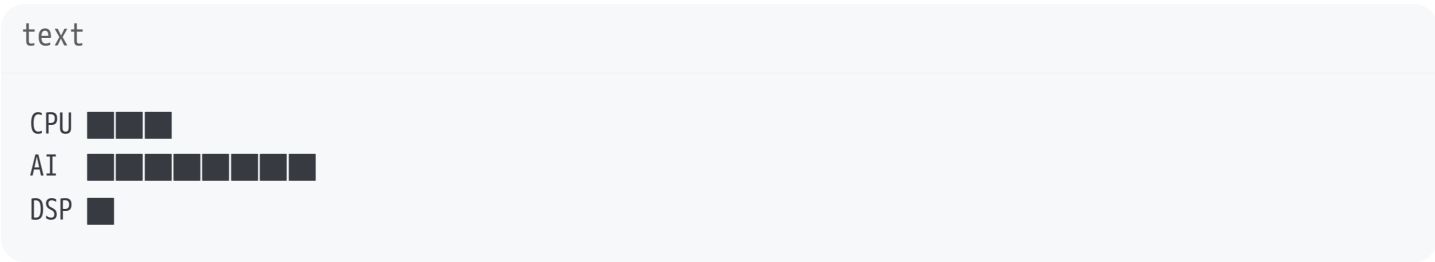
Resources become dynamically allocated.

Example:

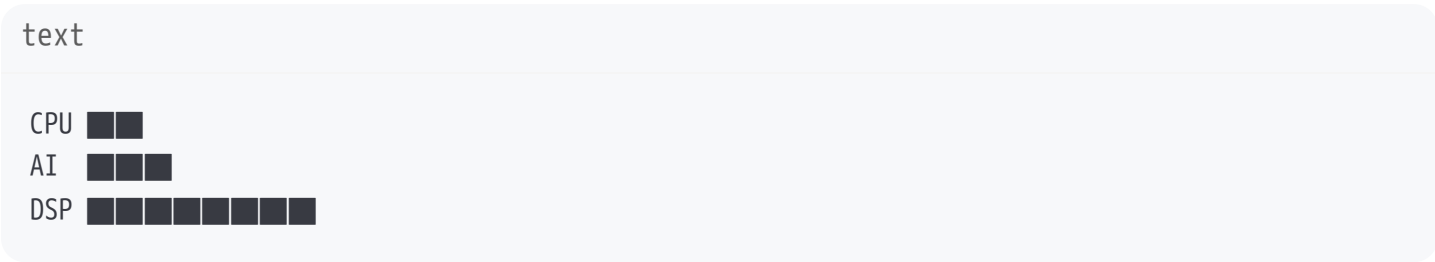
Normal



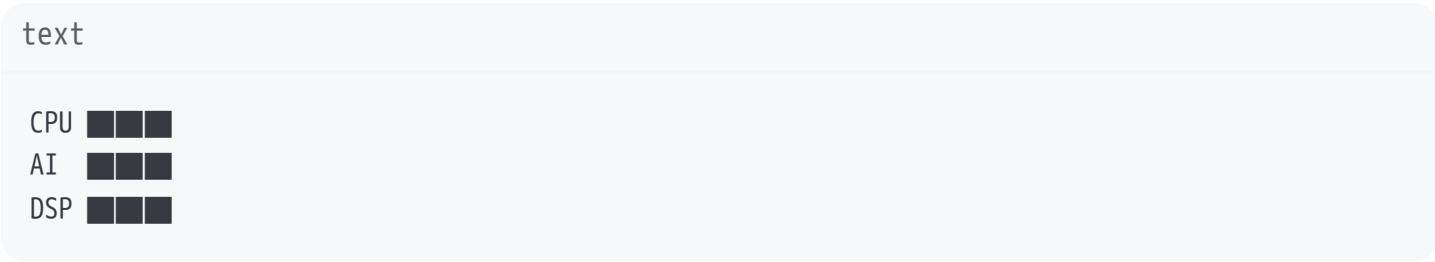
Then AI workload arrives:



Then signal-processing workload arrives:



And under thermal stress:



The architecture effectively becomes a **hardware economy**.

Compute resources are allocated where they create the most value.

6. Take this one step further: "hardware economics"

This could become one of our intellectual hooks.

Every hardware resource has:

```
text
energy cost
thermal cost
latency benefit
reliability impact
area cost
```

The policy engine treats them as competing resources.

For example:

```
text

AI accelerator:

+ very high performance
+ low latency
- high power
- thermal load

CPU:

+ flexible
+ low area
- slower
```

So the system makes a decision based on the current "price" of resources.

You could literally formulate:

$$Utility = w_p P + w_e E + w_r R + w_s S - w_t T$$

where the weights change according to the mission.

This gives the project a legitimate **architecture + algorithmic contribution**, rather than merely "we used reinforcement learning."

7. Add predictive adaptation

This is critical.

If adaptation only happens **after** something goes wrong:

```
text

temperature ↑
    ↓
react
```

that's reactive.

Instead:

text

```
temperature trend
  ↓
predict future state
  ↓
preemptively adapt
```

Example:

text

Current temperature: 72°C

Prediction:
87°C in 120 ms

Action:
reduce pipeline frequency
move workload
enable accelerator
reduce precision

So:

The chip doesn't react to failure. It anticipates it.

That is substantially more compelling.

8. Add graceful degradation

This could have enormous industrial relevance.

Today's systems often have:

working / failed

We introduce:

continuous degradation.

Imagine a system with:

text

100% capability

Then hardware begins degrading.

Instead of dying:

text

100%

↓

97%

↓

91%

↓

83%

↓

72%

The system progressively sacrifices non-critical functions.

For example, an autonomous vehicle:

text

Level 1:

full perception

Level 2:

reduce secondary processing

Level 3:

disable entertainment / non-critical workloads

Level 4:

increase redundancy

Level 5:

minimum safe operation

This is enormously relevant to:

- aerospace
- automotive
- medical systems
- industrial control
- defense

- satellites

And VLSID explicitly includes fault tolerance, reliability, functional safety, static/dynamic fault recoverability and automotive/EV systems in its scope. ([VLSID 2027](#))

9. Add "hardware immune memory"

Now combine self-healing with learning.

Every time the system encounters a problem:

```
text

event
  ↓
diagnosis
  ↓
response
  ↓
outcome
  ↓
store experience
```

Eventually:

```
text

"I have seen this pattern before."
```

So the system develops a **hardware experience database**.

Not a giant database.

Something tiny and hardware-friendly:

```
text

state signature
  ↓
best-known policy
  ↓
expected outcome
```

This gives us:

Experience-driven hardware adaptation.

That's much more interesting than simply calling it reinforcement learning.

10. Add federated learning between chips

This is where we can start talking about **industrial deployment**.

Imagine 10,000 industrial controllers.

Each one experiences slightly different conditions.

text

```
Machine 1 → experience
Machine 2 → experience
Machine 3 → experience
...
Machine N → experience
```

They don't need to send raw operational data.

They send:

text

```
policy updates
failure signatures
adaptation outcomes
```

to a central model.

Then:

text

```
Global model
  ↓
better policy
  ↓
distributed back to machines
```

Now the architecture becomes:

A fleet of self-improving machines.

That's a potentially enormous industrial story.

11. The truly wild idea: hardware evolution

This is the direction I'd keep in the back of our minds.

Instead of:

adaptive configuration

eventually:

adaptive architecture.

Suppose we have a reconfigurable fabric containing:

text

ALUs
MACs
DSP blocks
memory blocks
interconnect
specialized accelerators

The system can rearrange them.

So over time:

text

Day 1
general-purpose architecture

Day 2
AI-heavy architecture

Day 30
signal-processing-heavy architecture

Day 100
custom architecture optimized for actual workload

The physical chip hasn't changed.

But the **effective architecture has evolved.**

That is the seed of:

"Evolutionary Silicon."

12. Now imagine the industry impact

This is where your original requirement gets interesting.

Current semiconductor economics basically work like:

text

```
graph TD; A[identify market] --> B[design chip]; B --> C[tapeout]; C --> D[manufacture]; D --> E[ship]; E --> F[hardware remains fixed];
```

The problem is enormous:

Workloads change faster than silicon.

An AI chip designed today might encounter completely different models two years later.

EvoSoC's proposition:

text

```
graph TD; A[design generalized adaptive silicon] --> B[deploy]; B --> C[observe real workload]; C --> D[adapt]; D --> E[learn]; E --> F[continue adapting];
```

Potentially:

The useful lifetime of a chip becomes longer because the architecture evolves around the workload.

That's a semiconductor-industry-level argument.

13. We can even attack the "tapeout problem"

This is crazy, but worth exploring.

Imagine chip designers today spend enormous effort predicting:

"What will users want five years from now?"

Instead:

EvoSoC philosophy

Design a **reconfigurable computational substrate**.

Then let deployment data determine which configurations are optimal.

The chip becomes somewhat like:

a hardware platform rather than a fixed product.

This could eventually reduce:

- architecture redesign
- product fragmentation
- specialized ASIC variants
- hardware obsolescence
- some NRE
- time-to-market

We absolutely **cannot claim those benefits without evidence**, but this is the long-term industrial thesis.

14. Add security

Now EvoSoC becomes even more interesting.

A malicious workload could attempt:

text

```
thermal attack
power attack
fault injection
resource exhaustion
```

Our system detects abnormal behavior and adapts.

But there's another possibility:

Security becomes adaptive.

Example:

text

Normal:
standard security mode

Suspicious:
increase monitoring

Attack detected:
isolate subsystem

Severe attack:
reconfigure execution path

Recovery:
restore normal configuration

So the same architecture becomes:

performance manager + reliability manager + security manager.

15. Add "trust"

This is particularly interesting because the VLSID CFP explicitly mentions ensuring trust using untrusted tools/IP/models/manufacturing. ([VLSID 2027](#))

Imagine the AI policy itself is potentially wrong.

Therefore:

Never allow AI to directly control critical hardware.

Instead:

text

```
graph TD
    AI[AI proposes] --> SM[Safety monitor]
    SM --> FC[Formal constraints]
    FC --> HC[Hardware controller]
    HC --> EXEC[execute]
```

The AI becomes:

advisor

not:

authority.

That's a very defensible architecture.

16. We could create a "constitutional hardware layer"

This is a slightly crazy name, but the concept is excellent.

The chip has immutable rules:

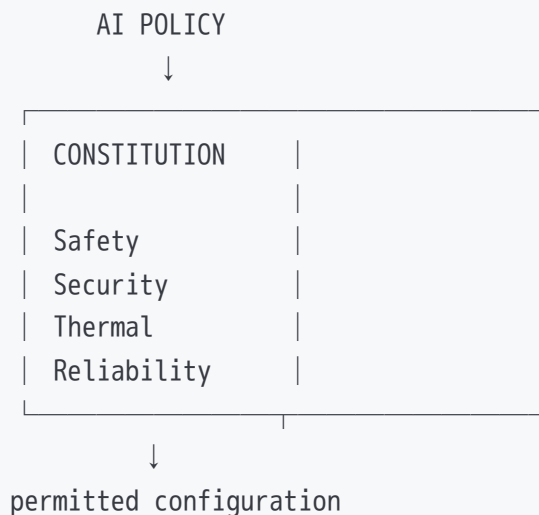
text

```
NEVER exceed thermal limit
NEVER disable safety core
NEVER violate memory isolation
NEVER exceed voltage boundary
NEVER reduce critical-task reliability
```

The AI can optimize everything **inside those boundaries**.

So:

text



This gives us:

constrained autonomous hardware.

That's much more credible than "AI controls the chip."

17. Put a digital twin beside it

Another major expansion.

Create a lightweight model of:

text

```
chip
+
workload
+
thermal state
+
reliability
```

The EvoSoC uses this model to simulate:

"If I change configuration X, what happens?"

Then:

text

```
real hardware
  ↓
state
  ↓
digital twin
  ↓
simulate possible actions
  ↓
choose best action
  ↓
real hardware
```

Now we have:

predictive + adaptive + self-aware computing.

And digital twins are explicitly within the VLSID intelligent-systems scope. ([VLSID 2027](#))

18. The ultimate architecture

At this point I'd stop calling it merely "EvoSoC."

I'd define a new architecture:

EvoCompute

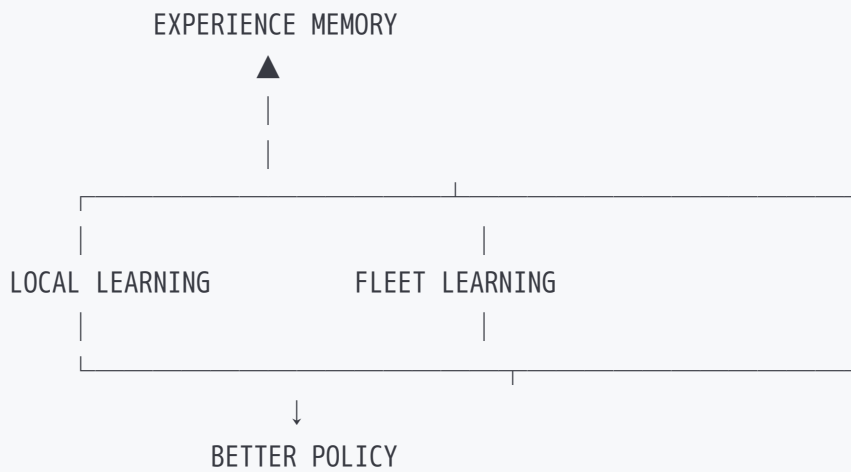
A self-evolving computing platform.

text



And then:

text



That's starting to look like an actual **new systems architecture**, not a student RISC-V project.

And here's the really important part

We shouldn't try to implement **all of this**.

That would become an unfinishable PhD.

Instead, we define the **full vision**, then build a carefully chosen demonstrator.

Version 1 — what we actually build

I'd choose:

Core

RV32IM

Adaptive resources

- configurable cache
- configurable pipeline/clock mode
- configurable accelerator
- dynamic resource allocation

Perception

- workload monitor
- thermal proxy
- timing-margin proxy

- fault injector

Intelligence

Tiny ML/policy engine.

Prediction

Short-horizon workload/thermal prediction.

Safety

Hardware-enforced constraints.

Recovery

Fault detection + graceful degradation.

Learning

Experience table.

That's enough.

And then demonstrate 4 industries

This is the part I'd really push.

Rather than claiming:

"EvoCompute is useful everywhere."

Build **four simulated deployment profiles**.



Automotive

Objective:

Safety + latency



Industrial robot

Objective:

throughput + predictive reliability



Satellite

Objective:

radiation resilience + energy



Medical edge device

Objective:

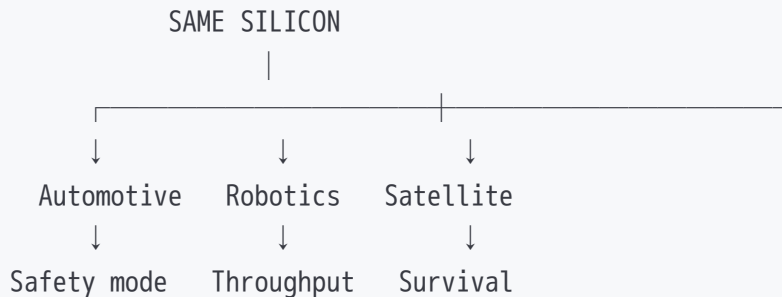
reliability + low power

Same hardware.

Different missions.

And show:

text



That is the killer demonstration.

The thesis becomes:

One adaptive architecture can transform itself into different application-specific machines without changing the underlying silicon.

That is substantially more disruptive than "we made a processor that optimizes itself."

The ultimate research question

I'd frame our entire project around one question:

Can a single piece of silicon continuously transform its effective architecture to optimize itself for changing workloads, physical conditions, reliability requirements, and mission objectives?

If we can answer **yes**, with hardware and measured data, we've got something genuinely interesting.

And it aligns almost suspiciously well with what VLSID is asking for: UDT specifically wants original methodologies/prototypes with measurable productivity, performance, power, quality, reliability, scalability or cost impact, while the 2027 theme is explicitly **AI → Physical Intelligence**. ([VLSID 2027](#))

But I think we can go one level further.

The next conceptual leap I'd explore is:

What happens when multiple EvoCompute nodes cooperate?

Not just:

one chip adapts itself.

But:

a network of chips dynamically reallocates computation across itself.

Then you get:

self-organizing compute infrastructure.

A node becomes overloaded → another node takes the workload.

A node becomes thermally stressed → computation migrates.

A node begins failing → the network reroutes.

A new workload appears → the network restructures.

Now the architecture isn't merely **self-adaptive**.

It's **self-organizing**.

And *that* could potentially take us from:

EvoSoC → EvoCompute → autonomous computing infrastructure.

That's the direction I'd explore next before we lock the architecture.

2026-10-05 12:31:51 You 

Okay quick question, what happens when I submit the idea tomorrow. Do I get selected as a fellow? Or something?

 ChatGPT gpt-5-6

No — **submitting the idea does not mean you become a fellow.**

For the **VLSID UDT (University Design Track)**, the process is essentially a **submission → review → selection → presentation/showcase** pipeline. The UDT is a conference track, not a fellowship program. ([Invalid URL](#))

So if you submit EvoCompute tomorrow:

1. **You submit the proposed work/idea** according to the UDT call.

2. **The organizers/reviewers evaluate it.**
3. If selected, you get an opportunity to **present/showcase the work at VLSID.**
4. You and your team would participate as **UDT participants/authors**, not "fellows."
5. Selection is **not the same as acceptance of a completed research paper** — UDT is specifically intended to surface promising university designs, including work-in-progress.

More importantly for us

I would **not submit EvoCompute tomorrow in its current conceptual form.**

Right now we have something potentially very ambitious:

self-aware + self-healing + predictive + morphable + mission-driven + learning + distributed computing.

That's the **vision.**

Before submitting, we should turn it into a sharply defined **UDT proposal** with:

Problem → Novel architecture → Specific technical innovations → Prototype plan → measurable metrics → industry demonstrator → expected impact.

And this is actually good news: **we don't need to have the entire thing fabricated tomorrow.** The point is to convince the reviewers that the proposed design is original, technically credible, and worth showcasing/developing.

So I would treat tomorrow's submission as:

"Here is a radically new architecture we are proposing and actively developing."

—not—

"Here is a finished CPU we already built."

And if we get selected, **that's when we go absolutely insane with the implementation.**

2026-10-05 13:06:40 You 

Okay numbers, if I want to get selected for this conference, I need to show numbers. What this project could save, how much can it change, what's the selling point, what are the pros, what are the cons, how much do pros outweigh the cons, how much friction is there if tomorrow this idea is going to be implemented in the industry.



I've got enough to see the central economic argument: **EvoCompute should sell avoided silicon redesign and wasted compute, not merely "better CPU performance."** I'm now stress-testing that claim against real industry constraints.

Yes. This is exactly the analysis we need before submitting, because the **idea itself is only half the selection battle**. The other half is proving that it matters economically, technically, and industrially.

I dug into the current VLSID UDT brief, recent adaptive-computing work, semiconductor economics, automotive/industrial architecture trends, and the latest reliability work. The picture is actually quite encouraging — but it also exposes where EvoCompute could fail if we design it badly. ([VLSID 2027](#))

1. First, the real thesis

I would now define the project as:

EvoCompute

Mission-adaptive, self-aware, self-optimizing and fault-resilient computing architecture

The core proposition:

One piece of silicon can continuously change its effective architecture according to workload, energy availability, thermal state, reliability, security and mission requirements — instead of being statically optimized at design time.

That is much stronger than:

"RISC-V processor with AI."

The industrial promise is:

today

one workload → one optimized hardware product

EvoCompute

many workloads + changing conditions → one adaptive hardware platform

2. What problem are we actually solving?

There are **five huge inefficiencies** in conventional silicon.

Problem	Conventional approach	EvoCompute response
Workloads change	Overprovision hardware	Reconfigure resources
Physical conditions change	Fixed safety margins	Sense + predict + adapt
Hardware faults occur	Reset / failover / redundancy	Detect + isolate + recover
Product requirements evolve	New silicon revision	Reuse adaptive platform
Compute resources sit idle	Static allocation	Dynamically trade resources

This is not a theoretical annoyance.

The semiconductor industry is already dealing with enormous design complexity. McKinsey/IBS estimates put chip-design cost around *50M at 28nm*, **104M at 16 nm**, *298M at 7nm* and **540M at 5 nm**. Those figures include architecture, verification, physical design, software, prototyping and validation. ([McKinsey & Company](#))

At the same time, Arm is explicitly pushing modular/chiplet architectures as a response to rising AI workload diversity, power constraints, design complexity and time-to-market pressure; Arm says some partners have reported **up to 12 months faster silicon delivery** and millions in NRE savings using its pre-integrated compute approach. ([Arm Newsroom](#))

So our project is entering a **real industry problem**, not inventing a problem to justify the chip.

3. The biggest selling point

I think we should stop selling:

"higher performance and lower power."

That's boring.

Every semiconductor presentation says that.

Our primary selling point should be:

"One silicon. Many optimal machines."

A customer doesn't buy EvoCompute because it is 11% faster.

They buy it because:

the same silicon can serve multiple product configurations and keep adapting after deployment.

That has implications for:

SKU count → engineering effort → inventory → NRE → product lifetime → energy → reliability → upgradeability.

That's the big story.

4. The economics of silicon reuse

Here's where the numbers become interesting.

Suppose a company needs three related products.

Conventional approach

text

Product A → ASIC A

Product B → ASIC B

Product C → ASIC C

Even with shared IP, every distinct silicon derivative incurs substantial architecture, verification, physical-design, validation and software work.

At 7 nm, a *full* chip-design program has been estimated around $298M$ * *; *at 5nm, about* * * **540M**.
(McKinsey & Company)

We should **not** claim EvoCompute saves \$298M automatically. That's dishonest.

Instead, use sensitivity analysis.

Suppose EvoCompute costs 15% extra NRE.

At 7 nm:

additional adaptive overhead

≈ \$44.7M

Now suppose the adaptive architecture lets the company avoid just **25% of another full-chip design effort**:

≈ \$74.5M avoided

Net:

≈ \$29.8M potential NRE advantage

And that's before counting energy, service life or downtime.

At 50% of a second design:

$149M \text{ avoided} - 44.7M \text{ adaptive overhead} = \sim \$104M \text{ net.}$

At a full avoided design:

$298M - 44.7M \approx \$253M.$

These are **scenario models**, not guaranteed savings. But this is exactly how we should frame the business case.

5. The killer economic argument: break-even per chip

Let's say our extra adaptive silicon increases unit cost.

That sounds bad.

But compare it with the avoided engineering cost.

For example, if the net NRE benefit is \$29.8M:

Production volume	NRE value per unit
1M units	\$29.80
5M	\$5.96
10M	\$2.98
50M	\$0.60
100M	\$0.30

This is why I think the **platform argument is potentially more powerful than the energy argument**.

At enough volume, even relatively small per-unit overhead becomes irrelevant compared with avoided engineering expenditure.

6. But why would a company pay for more silicon?

This is the first major counterargument.

Adaptive hardware isn't free.

We may need:

- spare compute resources
- reconfigurable interconnect
- monitoring circuitry
- state management
- policy engine

- memory for policies
- safety enforcement
- additional verification

So we could end up with:

text

Conventional:
small + efficient + fixed

EvoCompute:
larger + more complex + adaptable

That is **our biggest architectural risk**.

Therefore our project must prove:

The adaptation benefit exceeds the silicon overhead.

I'd set an initial project target of:

$\leq 10\%$ area overhead

for the adaptation/control fabric.

And I'd want:

$\geq 15\%$ average energy reduction

across representative dynamic workloads.

That gives us a simple proposition:

Spend $\leq 10\%$ more silicon to save $\geq 15\%$ energy while gaining resilience and product flexibility.

Whether we achieve those numbers is for the experiment to decide.

7. Is 15% energy reduction realistic?

We can't claim it yet.

But the idea is technically plausible because the ingredients already have strong evidence individually.

For example:

A 2024 IEEE study of learning-based DVFS for edge/cloud inference reported **33% average energy reduction**, with latency reductions of roughly **28.6–59.1%** under its tested scenarios. ([IEEE Xplore](#))

A 2026 IEEE edge-AI design combining approximate computing, DVFS and fine-grained gating reported up to:

- **50% dynamic power reduction**
- **33% static-power reduction**
- **24% area reduction**

with a measured accuracy tradeoff. ([IEEE Xplore](#))

And a 2025 IEEE timing-prediction design on a 28 nm RISC-V processor reported up to:

- **56.8% performance improvement**
- **28% power reduction**
- **3.79% area cost.** ([IEEE Xplore](#))

These don't prove EvoCompute.

They prove something important:

The individual knobs we want to unify are capable of creating double-digit gains.

Our research question is whether a **single intelligent controller can coordinate those knobs dynamically** and outperform static policies.

That's a much better research contribution.

8. Convert the power number into something a CEO understands

Suppose a compute subsystem consumes **20 W** continuously.

Our target:

15% energy reduction

= 3 W saved.

Per device:

26.28 kWh/year

Over five years:

131.4 kWh/device

At an illustrative electricity value of \$0.10/kWh:

≈ \$13.14/device over five years

That doesn't sound amazing.

But at:

100,000 devices

that's:

13.14 GWh

and approximately:

\$1.31M electricity value

At one million devices:

131.4 GWh / ~\$13.1M

Again, those are scenario calculations, not forecasts.

And this is why I **would not make energy our primary business case.**

9. Energy becomes huge at infrastructure scale

The IEA's latest outlook estimates global data-center electricity consumption at around **485 TWh in 2025**, rising to about **950 TWh in 2030**. ([IEA](#))

Now imagine — purely as an upper-bound thought experiment — an adaptive architecture eventually influenced just **5%** of the relevant electricity consumption and reduced it by 10%.

That theoretical amount would be:

$950 \text{ TWh} \times 5\% \times 10\% = 4.75 \text{ TWh/year}$

At \$0.10/kWh:

~\$475 million/year

Again:

do not put "\$475M savings" on the UDT slide as a claimed result.

That's an opportunity-envelope calculation showing why energy efficiency at infrastructure scale matters.

10. Reliability is where EvoCompute gets really powerful

The energy story gets us interested.

Reliability makes the architecture valuable.

There is already very recent evidence here.

A 2026 IEEE Transactions on Nuclear Science paper on self-healing RISC-V using dynamic partial reconfiguration reported:

- **95.34% non-disruptive recovery**
- **99.94% total recovery**

under its tested fault-injection/proton-irradiation scenarios. ([ResearchGate](#))

This is fantastic for us for two reasons.

Good news

The concept of hardware self-recovery is demonstrably viable.

Bad news

We **cannot** pitch:

"We invented self-healing RISC-V."

Someone just published it.

Our differentiator therefore must be:

EvoCompute does not only recover from faults. It continuously optimizes architecture across workload, physical state, power and reliability objectives.

That's an important distinction.

11. Downtime is potentially our biggest ROI lever

This is where industrial customers start listening.

Siemens' 2024 analysis reported approximately:

- **\$1.4T/year** of unplanned-downtime losses across the world's 500 largest companies
- roughly **11% of revenue**
- approximately **27 hours/month** of unplanned downtime at the large plants surveyed
- automotive production downtime around **\$2.3M/hour** in the surveyed large plants. ([Siemens Blog](#))

ABB's 2025 survey of 3,600 industrial decision-makers reported that **83%** considered unplanned downtime to cost at least 10,000/hour, while 76,500,000/hour. ([ABB Group](#))

But here's an important correction:

EvoCompute cannot claim it prevents factory downtime.

Not directly.

A broken gearbox isn't magically repaired by a RISC-V controller.

What we can claim is:

EvoCompute reduces the probability and impact of compute/control-system failures and can maintain degraded functionality when faults occur.

That is defensible.

12. The six-minute argument

This is a beautiful business-case illustration.

At the Siemens automotive benchmark:

\$2.3M/hour

So:

6 minutes $\approx$ \$230,000

That means a compute platform that prevents or safely mitigates **even a few minutes of causally attributable production interruption** can justify a surprisingly expensive silicon platform.

But again:

It only counts if our hardware is actually responsible for preventing that interruption and the production loss truly would have occurred.

This distinction will make us look much more mature than a typical student project.

13. Predictive maintenance becomes a secondary benefit

Once EvoCompute understands its own state, it can generate:

text

```
HEALTH = 81%  
TIMING MARGIN = declining  
THERMAL STRESS = elevated  
MEMORY ERROR RATE = increasing  
PREDICTED FAILURE RISK = 17%
```

That turns hardware monitoring into a maintenance signal.

IBM cites research indicating predictive maintenance can reduce overall maintenance costs by around **18–31%** versus traditional methods. (IBM)

Again, we shouldn't claim EvoCompute itself produces 18–31% savings.

Instead:

Our architecture exposes the hardware-health telemetry required for predictive operation and graceful degradation.

14. The really disruptive part: "mission economics"

Here's where I'd make the architecture fundamentally different.

A conventional chip optimizes for:

PPA.

EvoCompute optimizes for:

mission utility.

For example:

Industrial robot

text

priority:
throughput > energy > longevity

Drone

text

battery > latency > performance

Satellite

text

reliability > energy > performance

Medical device

text

safety > reliability > latency > energy

Autonomous vehicle

text

safety > deterministic latency > reliability > energy

Same silicon.

Different policy.

15. This creates a very powerful new abstraction

Today's hardware:

text

```
graph TD
    SPECIFICATION --> ARCHITECTURE
    ARCHITECTURE --> SILICON
    SILICON --> fixed_behaviour[fixed behaviour]
```

EvoCompute:

text

```
graph TD
    MISSION --> OBJECTIVES
    OBJECTIVES --> POLICY
    POLICY --> ARCHITECTURAL_CONFIGURATION[ARCHITECTURAL CONFIGURATION]
    ARCHITECTURAL_CONFIGURATION --> SILICON
    SILICON --> PHYSICAL_WORLD[PHYSICAL WORLD]
    PHYSICAL_WORLD --> FEEDBACK
    FEEDBACK --> MISSION
```

This is the intellectual heart of the project.

We're effectively saying:

"Architecture becomes a runtime variable."

That's the statement I'd want the reviewer remembering after they leave the room.

16. Resource trading

This is another important feature.

Imagine:

text

CPU
AI accelerator
DSP
SRAM
interconnect

Instead of all of them being permanently configured, EvoCompute continuously determines:

Where will the next joule of energy produce the most useful computation?

Example:

text

Normal workload

CPU ■■■■■■
AI ■■
DSP ■

AI workload arrives:

text

CPU ■■■
AI ■■■■■■
DSP ■

Signal-processing workload:

text

CPU ■■
AI ■■■
DSP ■■■■■■

Thermal crisis:

text

CPU ■■■
AI ■■■
DSP ■■■

That makes the silicon behave more like a **resource marketplace**.

This is a useful metaphor for the business story:

Compute resources are allocated where they create the highest mission value.

17. We should add a hardware "constitution"

This is not just cool branding.

It's necessary.

The ML/controller should **never have unrestricted authority**.

There should be immutable hardware constraints:

```
text

T < TMAX
V < VMAX
critical core cannot be disabled
safety task cannot lose deadline
memory isolation cannot be violated
minimum redundancy must be maintained
```

So:



This solves one of the biggest industrial objections:

"Why would I let an ML model control safety-critical silicon?"

Answer:

You don't. The ML controller proposes; immutable hardware constraints decide what is permissible.

18. This is where industrial friction becomes manageable

And this is extremely important.

We should **not deploy EvoCompute into a safety-critical automobile tomorrow.**

That's a terrible first market.

Why?

Automotive silicon requires things like:

- functional-safety processes
- qualification
- reliability testing
- bounded and explainable behavior
- extensive verification
- lifecycle management

AEC-Q100, for example, defines automotive IC qualification requirements and temperature grades extending to **-40°C to 150°C** for Grade 0 products. ([Texas Instruments](#))

The problem isn't that the architecture is impossible.

The problem is that **changing behaviour creates a verification explosion.**

19. The verification problem is probably our #1 technical risk

Suppose we have:

text

```
4 operating modes
3 voltage levels
3 frequency levels
3 cache modes
2 accelerator states
```

That's already:

$$4 \times 3 \times 3 \times 3 \times 2 = 216$$

architectural configurations.

Now multiply that by:

- temperature corners
- voltage corners
- workload classes
- fault states
- memory conditions

We quickly reach thousands of state combinations.

That's why the project needs something very clever:

adaptation must occur inside a formally bounded configuration space.

The AI isn't allowed to invent arbitrary hardware states.

It selects from:

```
text

CONFIG_0
CONFIG_1
CONFIG_2
...
CONFIG_N
```

where each configuration has already been formally validated.

This enormously reduces industrial friction.

20. That gives us "certifiable adaptation"

This could actually become another paper-level contribution.

Instead of:

arbitrary AI-controlled hardware

we propose:

AI-selected, formally certified architectural states.

Meaning:

```
text
```

OFFLINE
verification
↓
approved states
↓
production chip
↓
AI selects only
approved state

This is much more believable.

21. Industry-by-industry applicability

Here's how I'd rate the opportunity **relative to implementation friction**.

Industry	Impact	Deployment friction	Why
Industrial edge / robotics	★★★★★	★★☆☆☆	High downtime value, heterogeneous workloads
Drones / UAV	★★★★★	★★★★☆☆	Energy + compute + graceful degradation
Telecom edge	★★★★★ ☆	★★★★☆☆	Variable traffic/load
Automotive	★★★★★	★★★★★★	Huge value but brutal qualification
Aerospace/satellite	★★★★★	★★★★★★	Reliability enormous, volumes low
Datacenter	★★★★★	★★★★★☆☆	Huge power scale, but mature infrastructure
Medical	★★★★★	★★★★★★	Safety/regulatory burden
Consumer electronics	★★★★★ ☆	★★★★☆☆	Volume huge, margins brutal

My recommendation:

Industrial robotics / edge automation first.

Not autonomous driving.

Not medical.

Not satellites.

Not hyperscale data centers.

Those become **second-generation applications**.

22. Why industrial robotics is the sweet spot

It combines:

Huge benefit

Downtime is expensive.

Heterogeneous compute

Vision + control + communication + inference + DSP.

Dynamic conditions

Load, temperature, workload and timing can change.

Need for reliability

Failure can interrupt production.

No need for immediate automotive-grade certification

At least for a non-safety-critical supervisory/vision compute node.

Easy demonstration

We can literally build:

text

```
camera/sensor stream
  ↓
EvoCompute FPGA
  ↓
vision / control workload
  ↓
load changes
  ↓
architecture changes
  ↓
fault injection
  ↓
```

graceful degradation

That makes for a spectacular UDT demo.

23. The second market I'd attack: UAVs

This is an even better demonstration environment.

A drone continuously experiences:

text

```
battery ↓  
temperature ↑  
compute load changes  
communication changes  
mission priority changes
```

So it naturally needs:

dynamic tradeoffs between performance, energy, reliability and mission completion.

Imagine the drone decides:

"Battery is critically low."

The architecture morphs:

text

```
large model → small model  
high precision → lower precision  
CPU frequency ↓  
accelerator gating ↑  
noncritical compute OFF
```

But:

obstacle avoidance remains untouched.

That is exactly the kind of physical intelligence VLSID's 2027 theme is targeting. ([VLSID 2027](#))

24. Automotive becomes the long-term jackpot

The automotive industry is already moving toward centralized/zonal architectures.

McKinsey estimates zonal E/E architectures could reduce material costs by **10–20%**, reduce engineering expense and accelerate concept-to-production time by roughly **20–30%** in the relevant

architecture shift. (McKinsey & Company)

And centralized compute increases the importance of flexible, software-defined compute platforms. (McKinsey & Company)

So EvoCompute could eventually become:

the adaptive central compute platform inside a software-defined vehicle.

But again:

not our first deployment.

25. Aerospace is where self-healing becomes uniquely valuable

Space computers face single-event upsets and functional interruptions due to radiation.

NASA-supported work has specifically investigated fault-tolerant commercial processors for space environments, including autonomous recovery mechanisms. (NASA TechPort)

EvoCompute's future mode:

```
text

normal
  ↓
radiation event
  ↓
health deterioration
  ↓
isolate affected resource
  ↓
migrate computation
  ↓
reduce noncritical workload
  ↓
continue mission
```

That is a beautiful extension.

But again:

qualification friction = enormous.

26. Medical is interesting – but dangerous for us

There is a fascinating parallel here.

The FDA now explicitly provides guidance for **Predetermined Change Control Plans** for AI-enabled medical devices — essentially allowing manufacturers to pre-specify controlled ways in which an AI-enabled device can evolve while maintaining safety/effectiveness. ([U.S. Food and Drug Administration](#))

That's conceptually very close to what our hardware constitution does.

But it's also evidence that:

controlled adaptation is the right direction; uncontrolled adaptation is unacceptable.

So medical could eventually be a fantastic use case, but absolutely not our first product.

27. Data centers are the gigantic opportunity

The IEA expects global data-center electricity consumption to approach **950 TWh/year by 2030**. ([IEA](#))

If adaptive computing eventually achieved even a modest 5–10% efficiency improvement over a meaningful share of workloads, the absolute energy value could be enormous.

But data centers already have:

- sophisticated schedulers
- DVFS
- heterogeneous CPUs/GPUs
- virtualization
- workload migration
- accelerators
- power management

So the bar is extremely high.

We don't win by saying:

"we can dynamically scale compute."

They already do.

We win by moving adaptation **into the silicon architecture itself** and coordinating:

microarchitecture + reliability + power + thermal + workload + mission policy.

28. Our competition isn't just CPUs

This is crucial.

Our competitors are:

ASIC

Best performance/efficiency.

Weakness: fixed.

CPU/GPU

Flexible.

Weakness: potentially inefficient for specialized workloads.

FPGA

Very adaptable.

Weakness: reconfiguration/resources/power/performance tradeoffs.

Chiplets

Modular.

Weakness: still usually statically composed.

Dynamic power management

Efficient.

Weakness: generally narrower optimization scope.

Self-healing hardware

Reliable.

Weakness: primarily reliability-focused.

AI workload schedulers

Adaptive.

Weakness: often adaptation happens above the silicon.

EvoCompute's pitch:

bring all of these dimensions into a unified hardware-level feedback loop.

29. But there is a brutal novelty issue

Self-aware MPSoCs aren't new.

A 2024 survey specifically reviews autonomous MPSoC management involving:

- power

- thermal events
- fault tolerance
- security
- self-awareness
- goal management
- learning. (PubMed Central (PMC))

Self-adaptive computing is also a mature research area, with decades of work on self-configuration, self-optimization, self-healing and self-protection. (ScienceDirect)

Therefore our paper **cannot claim:**

"first self-aware processor."

It would be demolished in review.

30. Our novelty must be the unification

This is the distinction I would defend:

Existing systems typically optimize one axis:

text

```
power
OR
thermal
OR
faults
OR
workload
OR
reconfiguration
```

EvoCompute:

text

MISSION



- | unified state representation
- |
- | workload
- | power



The contribution becomes:

Mission-driven cross-domain runtime architecture optimization under hardware-enforced safety constraints.

That is a much stronger research statement.

31. What numbers should we actually promise?

This is very important.

Do not fabricate results.

Instead, on tomorrow's deck, call these:

Design targets

I'd put:

Metric	Target
Energy reduction	≥15%
Performance improvement on workload mismatch	≥20%
Performance/W improvement	≥25%
Area overhead	≤10%
Fault recovery	≥95% of injected recoverable faults
Critical-function survival	≥99.9% within tested fault model

Metric	Target
Adaptation decision latency	<10 μ s target
Unauthorized configuration states	0
Supported mission profiles	$\geq$ 4
Hardware utilization improvement	$\geq$ 20%

These are **targets, not claims**.

Then we build the prototype and replace them with measured values.

32. But I would make one of those even more ambitious

This:

one silicon → four industries

We create four mission profiles:

text

Industrial robot
UAV
Automotive edge compute
Satellite

And show that the same hardware architecture adopts different policies.

That's a killer result.

For example:

Mission	Primary objective	EvoCompute behavior
Robot	Throughput	More accelerator resources
UAV	Battery	Aggressive power scaling
Vehicle	Safety	Redundancy + deterministic timing
Satellite	Survival	Fault isolation + degradation

That demonstrates **generalization** rather than a benchmark-specific trick.

33. We need a baseline matrix

Our paper should compare:

B0 — static RISC-V

No adaptation.

B1 — conventional DVFS

Power adaptation only.

B2 — workload-aware scheduler

Workload adaptation.

B3 — fault-tolerant design

Reliability adaptation.

B4 — EvoCompute

Everything integrated.

Then:

Metric	B0	B1	B2	B3	EvoCompute
Energy	baseline	↓	↓	~	↓
Performance	baseline	~	↑	↓	↑
Recovery	poor	poor	poor	↑	↑ ↑
Adaptability	low	medium	medium	medium	high
Area	baseline	+	+	+	+
Mission optimization	no	no	partial	no	yes

This is much stronger experimentally.

34. The most important negative: adaptation overhead

Suppose a workload lasts only 10 μs.

If switching configuration takes 8 μs:

congratulations, we've saved nothing.

So the system needs:

adaptation economics

Before changing configuration:

$$Benefit_{future} > Cost_{switch}$$

The controller should ask:

"Will this configuration remain useful long enough to justify switching?"

That leads naturally to a predictive policy.

This is an actual research contribution.

35. The second negative: oscillation

Bad adaptive controllers can do this:

text

HIGH
↓
LOW
↓
HIGH
↓
LOW
↓
HIGH

every few milliseconds.

That's terrible.

So we need:

hysteresis

minimum residency time

confidence threshold

predicted workload horizon

switching cost model

That makes EvoCompute much more sophisticated.

36. Third negative: AI mistakes

The model might choose the wrong configuration.

Solution:

text

```
AI
↓
candidate configuration
↓
constraint engine
↓
admissible?
↓
performance/utility model
↓
execute
```

No free-form autonomous hardware.

37. Fourth negative: hardware area

We need to prove that:

spare capability < benefit obtained.

This is probably why I'd avoid full FPGA-style runtime reconfiguration initially.

Instead, use:

coarse-grained morphing

Examples:

- enable/disable accelerator
- cache ways
- pipeline mode
- clock domain selection
- precision mode
- redundant unit activation
- workload migration

This gets us most of the adaptive benefit without enormous reconfiguration complexity.

38. Fifth negative: verification explosion

Solved by:

certified configuration space.

Every mode is predefined.

The AI chooses between certified states.

This becomes:

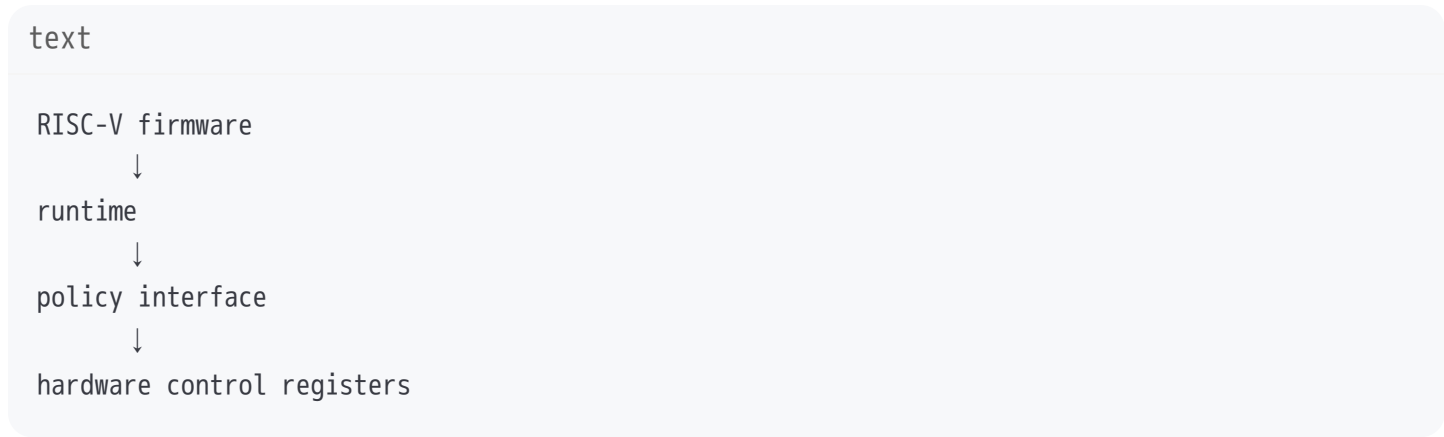
bounded autonomy

rather than unlimited autonomy.

39. Sixth negative: software integration

A chip doesn't exist in a vacuum.

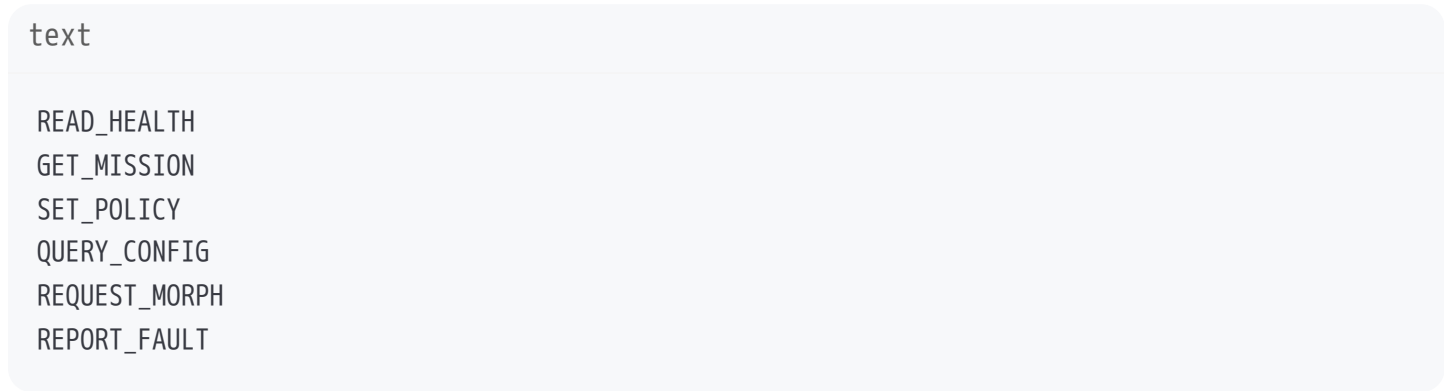
We'll need:



But this is actually an advantage.

RISC-V gives us freedom to define a clean custom interface.

Potential custom CSR instructions:



Now EvoCompute has a recognizable software interface.

40. Seventh negative: security

An attacker could manipulate:

text

temperature sensor
workload telemetry
fault status
policy inputs

and force expensive or unsafe states.

So we need:

- trusted sensors
- authenticated policy updates
- anomaly detection
- immutable safe-state controller
- protected configuration memory

This also brings the project directly into VLSID's hardware-security scope. (VLSID 2027)

41. The industrial friction score

My internal assessment for a **first-generation EvoCompute industrial edge product**:

Factor	Difficulty
RTL implementation	★★☆☆☆
FPGA prototyping	★★☆☆☆
Runtime controller	★★★★☆
ML policy	★★★★☆
Verification	★★★★☆
PPA optimization	★★★★☆
Software integration	★★★★☆
Industrial certification	★★★★☆
Automotive certification	★★★★★
Aerospace certification	★★★★★
Medical certification	★★★★★
Manufacturing integration	★★★★☆

Overall:

~3.5/5 implementation friction

That's actually a good place to be for a research project.

Extremely disruptive ideas often have 5/5 friction and never leave the PowerPoint.

42. Pros versus cons

Let's brutally score the actual proposition.

Upside

Benefit	Score
Workload adaptability	10/10
Product flexibility	10/10
Reliability	9/10
Energy optimization	8/10
Hardware lifetime	9/10
Industrial applicability	9/10
RISC-V compatibility	9/10
Research novelty potential	9/10
Long-term commercialization	9/10

Downsides

Cost/risk	Score
Area overhead	6/10
Verification complexity	9/10
Control stability	7/10
AI unpredictability	7/10
Software complexity	6/10
Certification	10/10
Manufacturing complexity	5/10

Cost/risk	Score
Implementation difficulty	8/10

My assessment:

Strategic upside: ~9.1/10

Engineering difficulty: ~7.5/10

That's exactly the sort of asymmetry I want.

Hard enough to be impressive.

Not impossible to prototype.

43. The real business model

We shouldn't sell the chip as:

"a CPU."

I'd sell it as:

Adaptive Compute Platform

Possible revenue layers:

Silicon

EvoCompute SoC.

Mission profiles

Automotive / robotics / UAV / industrial.

Runtime software

Policy/runtime management.

Optimization engine

Mission-specific tuning.

Fleet intelligence

Aggregated experience from deployed devices.

This eventually becomes:

text

```
hardware
+
runtime
+
adaptation policy
+
fleet learning
```

That's much more defensible than selling another processor.

44. The endgame: EvoMesh

This should **not** be in our core prototype, but it belongs in the vision.

Imagine 100 EvoCompute nodes:

```
text

Node A
Node B
Node C
...
Node 100
```

Node A overheats.

Its workload migrates.

Node C detects a failure.

Its workload gets redistributed.

Node F suddenly receives an AI-heavy workload.

The network reorganizes.

Now:

the computing infrastructure itself becomes self-organizing.

That's where we could eventually go from:

self-adaptive chip

to:

self-organizing compute infrastructure.

This is potentially much bigger than EvoSoC itself.

45. And then something even bigger happens

We can imagine the semiconductor industry moving through:

Generation 1

Fixed silicon

text

design once → deploy

Generation 2

Configurable silicon

text

design once → configure

Generation 3

Adaptive silicon

text

deploy → sense → optimize

Generation 4

Self-healing silicon

text

deploy → detect → recover

Generation 5

Self-evolving compute

text

deploy
↓
sense
↓
predict
↓
adapt
↓

```
learn
↓
reorganize
↓
repeat
```

I think **that** should be the philosophical ambition of the project.

46. What actually changes if this works?

Here's the strongest industrial statement I can make without overselling:

Today

Companies predict future workloads and build silicon accordingly.

With EvoCompute

Companies could design a broader adaptive silicon platform and let deployment conditions determine the optimal configuration.

That could eventually reduce:

- hardware specialization
- product fragmentation
- redesign frequency
- overprovisioning
- energy waste
- failure impact

while improving:

- resilience
- utilization
- flexibility
- product lifetime
- time-to-market.

The industry is already moving toward modularity and flexible compute because workload diversity is increasing; EvoCompute takes that flexibility **inside the runtime architecture**. ([Arm Newsroom](#))

47. What I think the UDT reviewer should see

The VLSID UDT specifically asks for **quantifiable impact on productivity, performance, power, quality, reliability, scalability, manufacturability, cost or time-to-market**, and it explicitly encourages autonomous/adaptive/reconfigurable architectures and cognitive/adaptive systems. ([VLSID 2027](#))

So our six-slide story should essentially be:

Slide 1

EvoCompute

One Silicon. Many Optimal Machines.

Slide 2 – The problem

text

Workloads change
Physical conditions change
Products change
Silicon doesn't

Then:

50M → 540M chip-design economics

950 TWh projected data-center electricity by 2030

\$1.4T industrial downtime exposure

with proper source labels. ([McKinsey & Company](#))

Slide 3 – The invention

text

Sense
↓
Understand
↓
Predict
↓
Constrain
↓
Morph
↓
Learn

Slide 4 – Why it's different

text

Performance
Energy
Thermal
Reliability
Security
Mission

one closed-loop optimization system

Slide 5 – Evidence

This is the crucial slide.

Eventually:

text

15-30% energy ↓
20%+ performance/W ↑
95%+ fault recovery
<10% area overhead
4 mission profiles

But only **actual measured values** get shown as results.

Slide 6 – Industry impact

text

ONE ADAPTIVE SILICON PLATFORM

|—— Industrial
|—— Robotics
|—— UAV
|—— Automotive
|—— Aerospace
|—— Edge AI

And the final line:

"From fixed-function silicon to mission-adaptive silicon."

48. My honest assessment

This is now much more than a "cool VLSI project."

I'd rate it:

Technical ambition

9.5/10

UDT relevance

10/10

Industry relevance

9/10

Research potential

9/10

Prototype feasibility

8/10

Commercial potential

9/10

Current novelty without refinement

6.5/10

Novelty after we define the architecture properly

9/10 potential

The last part matters enormously.

The general idea of adaptive/self-aware computing already exists. ([ScienceDirect](#))

Our job is to invent a **specific architecture, control mechanism, configuration space, safety model, objective function and evaluation methodology** that makes EvoCompute different enough to stand on its own.

And I think the strongest formulation we've reached so far is:

EvoCompute = mission-driven, predictive, cross-domain hardware adaptation constrained by a hardware-enforced constitution.

That is the version I would take to VLSID.

One final practical point: **do not put our giant long-term vision on the first submission as though we've already built it.** UDT explicitly accepts work-in-progress, prototypes and late-breaking developments, so we can honestly submit the ambitious architecture while clearly distinguishing **proposed architecture, measurable targets, and demonstrated results.** ([VLSID 2027](#))

The next thing I'd do is turn all of this into a **single-page "EvoCompute technical/business specification"**: exact blocks, exact novelty claims, exact equations, exact target numbers, exact baselines, exact experiments, exact industry use-case, and exact claims we are legally/scientifically safe to put in tomorrow's six-slide blind submission. That becomes the blueprint for the entire project.